

```

import streamlit as st

from groq import Groq

from dotenv import load_dotenv

from datetime import date, datetime

import os

import json

import re


# =====

# PAGE CONFIG

# =====

st.set_page_config(

    page_title="StudyFlow AI",

    page_icon="📅",

    layout="wide"

)


# =====

# LOAD GROQ API

# =====

load_dotenv()

API_KEY = os.getenv("GROQ_API_KEY")

if not API_KEY:

    st.error("GROQ_API_KEY not found. Please add it to your .env file.")

    st.stop()

```

```
client = Groq(api_key=API_KEY)
```

```
# =====
```

```
# CUSTOM CSS
```

```
# =====
```

```
st.markdown("""
```

```
<style>
```

```
.stApp {
```

```
    background: linear-gradient(
```

```
        135deg,
```

```
        #f7f5ff,
```

```
        #eef2ff,
```

```
        #fff5fb
```

```
    );
```

```
}
```

```
[data-testid="stSidebar"] {
```

```
    background: linear-gradient(
```

```
        180deg,
```

```
        #17143b,
```

```
        #2b1b55,
```

```
        #431b63
```

```
    );
```

```
}
```

```
[data-testid="stSidebar"] * {
```

```
    color: white !important;
```

```
}
```

```
.hero-title {
```

```
    font-size: 42px;
```

```
    font-weight: 800;
```

```
    color: #241c54;
```

```
}
```

```
.hero-subtitle {
```

```
    font-size: 17px;
```

```
    color: #6b7280;
```

```
    margin-bottom: 25px;
```

```
}
```

```
.card {
```

```
    background: white;
```

```
    padding: 22px;
```

```
    border-radius: 20px;
```

```
    margin: 10px 0;
```

```
    box-shadow: 0 8px 25px rgba(60,50,120,0.08);
```

```
    border: 1px solid #ececff;
```

```
}
```

```
.ai-card {
```

```
    background: linear-gradient(
```

```
        135deg,
```

```
        #6d28d9,
```

```
        #9333ea,
```

```
        #db2777
```

```
);  
color: white;  
padding: 28px;  
border-radius: 22px;  
margin: 20px 0;  
}  
.task-card {  
background: white;  
border-left: 6px solid #7c3aed;  
padding: 18px;  
border-radius: 16px;  
margin: 12px 0;  
box-shadow: 0 5px 20px rgba(60,50,120,0.07);  
}  
.activity-card {  
background: white;  
border-left: 6px solid #ec4899;  
padding: 18px;  
border-radius: 16px;  
margin: 12px 0;  
}  
.section-title {  
font-size: 27px;  
font-weight: 800;  
color: #241c54;
```

```
}  
  
.stButton > button {  
    border-radius: 12px;  
    background: linear-gradient(  
        90deg,  
        #6d28d9,  
        #9333ea  
    );  
    color: white;  
    font-weight: 700;  
    border: none;  
}
```

</style>

```
""", unsafe_allow_html=True)
```

```
# =====
```

```
# SESSION STATE
```

```
# =====
```

```
if "tasks" not in st.session_state:
```

```
    st.session_state.tasks = []
```

```
if "activities" not in st.session_state:
```

```
    st.session_state.activities = []
```

```
if "generated_plan" not in st.session_state:
```

```
    st.session_state.generated_plan = None
```

```
if "quiz" not in st.session_state:
```

```

    st.session_state.quiz = []

if "quiz_score" not in st.session_state:

    st.session_state.quiz_score = None

if "quiz_submitted" not in st.session_state:  st.session_state.quiz_submitted = False

if "assistant_question" not in st.session_state:

    st.session_state.assistant_question = ""

if "assistant_answer" not in st.session_state:

    st.session_state.assistant_answer = None


# =====

# GROQ FUNCTION

# =====

def set_assistant_question(question):

    st.session_state.assistant_question = question

def ask_groq(prompt, json_mode=False):

    try:

        request_options = {}

        if json_mode:

            request_options["response_format"] = {

                "type": "json_object"

            }

        response = client.chat.completions.create(

            model="openai/gpt-oss-120b",

            messages=[

                {

```

```
"role": "system",
```

```
"content": ""
```

You are StudyFlow AI.

You are an intelligent student productivity assistant.

Your job is to:

- create realistic study tasks
- create study activities
- organize time
- create revision plans
- help students prepare for exams
- improve productivity

Keep plans practical for college students.

Do not overload the student.

```
""
```

```
},
```

```
{
```

```
"role": "user",
```

```
"content": prompt
```

```
}
```

```
],
```

```
temperature=0.2 if json_mode else 0.7,
```

```
max_completion_tokens=4000,
```

```
**request_options
```

```
)
```

```
return response.choices[0].message.content or "
```

```
except Exception as e:
```

```
    return f"AI Error: {str(e)}"
```

```
# =====
```

```
# SIDEBAR
```

```
# =====
```

```
with st.sidebar:
```

```
    st.markdown(
```

```
        """
```

```
        <div style="text-align:center;padding:20px;">
```

```
            <div style="font-size:55px;">❏</div>
```

```
            <h2>StudyFlow AI</h2>
```

```
            <p>AI Student Productivity</p>
```

```
        </div>
```

```
        """,
```

```
        unsafe_allow_html=True
```

```
    )
```

```
st.divider()
```

```
menu = st.radio(
```

```
    "MENU",
```

```
    [
```

```
        "❏ Dashboard",
```

```
        "❏ AI Productivity",
```

```
        "❏ Tasks",
```

```
        "❏ Activities",
```

```

        "☐ AI Quiz",
        "☐ AI Assistant"
    ]
)

# =====
# DASHBOARD
# =====

if menu == "☐ Dashboard":
    st.markdown(
        '<div class="hero-title">Good to see you! ☐</div>',
        unsafe_allow_html=True
    )
    st.markdown(
        '<div class="hero-subtitle">'
        'Your AI-powered student productivity dashboard.'
        '</div>',
        unsafe_allow_html=True
    )
    total_tasks = len(st.session_state.tasks)
    completed_tasks = sum(
        task["completed"]
        for task in st.session_state.tasks
    )
    pending_tasks = total_tasks - completed_tasks

```

```
total_activities = len(
    st.session_state.activities
)
progress = (
    int((completed_tasks / total_tasks) * 100)
    if total_tasks > 0
    else 0
)
c1, c2, c3, c4 = st.columns(4)
c1.metric(
    "☐ Total Tasks",
    total_tasks
)
c2.metric(
    "☐ Completed",
    completed_tasks
)
c3.metric(
    "☐ Pending",
    pending_tasks
)
c4.metric(
    "☐ Activities",
    total_activities
)
```

```

st.markdown(
    '<div class="section-title">☐ Productivity Progress</div>',
    unsafe_allow_html=True
)

st.progress(progress / 100)

st.write(
    f"***{progress}% tasks completed***"
)

st.markdown(
    """
    <div class="ai-card">

        <h2>☐ Your AI Productivity Partner</h2>

        <p>

            Generate personalized tasks, activities,

            schedules and study plans using AI.

        </p>

    </div>

    """,
    unsafe_allow_html=True
)

st.markdown(
    '<div class="section-title">☐ AI Features</div>',
    unsafe_allow_html=True
)

```

```
c1, c2, c3 = st.columns(3)
```

```
with c1:
```

```
    st.markdown(
        """
        <div class="card">

            <h3>📌 AI Productivity</h3>

            <p>

                Generate complete study work automatically.

            </p>

        </div>
        """,
        unsafe_allow_html=True
    )
```

```
with c2:
```

```
    st.markdown(
        """
        <div class="card">

            <h3>📌 AI Quiz</h3>

            <p>

                Generate MCQs and test your knowledge.

            </p>

        </div>
        """,
        unsafe_allow_html=True
    )
```

```

with c3:

    st.markdown(
        """

        <div class="card">

            <h3>📦 AI Assistant</h3>

            <p>

                Ask questions and learn difficult concepts.

            </p>

        </div>

        """,
        unsafe_allow_html=True
    )

# =====

# AI PRODUCTIVITY

# =====

elif menu == "📦 AI Productivity":

    st.markdown(
        '<div class="hero-title">📦 AI Productivity Generator</div>',
        unsafe_allow_html=True
    )

    st.write(
        "Tell AI what you need to study and it will create your work automatically."
    )

```

```

st.markdown(
    """
    <div class="ai-card">

        <h2>☐ Let AI organize your study work</h2>

        <p>

            AI will generate tasks, activities, time blocks,

            breaks and productivity tips.

        </p>

    </div>

    """,
    unsafe_allow_html=True
)

c1, c2 = st.columns(2)

with c1:

    subject = st.selectbox(
        "☐ Subject",
        [
            "Python",
            "DBMS",
            "Data Structures",
            "Artificial Intelligence",
            "Machine Learning",
            "Data Science",
            "Operating System",
            "Computer Networks",

```

```
        "Web Development"
    ]
)
topic = st.text_input(
    "□ Topic",
    placeholder="Example: Python Functions"
)
goal = st.selectbox(
    "□ Goal",
    [
        "Exam Preparation",
        "Learning Concept",
        "Revision",
        "Assignment",
        "Project",
        "Interview Preparation"
    ]
)
with c2:
    difficulty = st.selectbox(
        "□ Difficulty",
        [
            "Beginner",
            "Intermediate",
            "Advanced"
```

```

    ]
)
available_hours = st.slider(
    "☐ Available Study Hours",
    1,
    12,
    3
)
deadline = st.date_input(
    "☐ Deadline",
    min_value=date.today()
)
if st.button("☐ Generate My Productivity Plan"):
    if not topic.strip():
        st.warning(
            "Please enter a topic first."
        )
    else:
        prompt = f"""

```

Create a personalized productivity plan for a college student.

Subject:

{subject}

Topic:

{topic}

Goal:

{goal}

Difficulty:

{difficulty}

Available study time:

{available_hours} hours

Deadline:

{deadline}

Generate ONLY valid JSON.

Use this exact format:

```
{{
```

```
  "tasks": [
```

```
    {{
```

```
      "name": "Task name",
```

```
      "description": "Short description",
```

```
      "priority": "High",
```

```
      "duration": 45,
```

```
      "category": "Study"
```

```
    }}
```

```
  ],
```

```
  "activities": [
```

```
    {{
```

```
      "name": "Activity name",
```

```
      "duration": 30,
```

```
      "type": "Study"
```

```
    }}
```

```

],
"schedule": [
    {{
        "time": "9:00 AM - 9:45 AM",
        "work": "Study topic"
    }}
],
"tips": [
    "Productivity tip 1",
    "Productivity tip 2"
]
}}

Rules:

1. Create realistic tasks.
2. Create practical study activities.
3. Total study time should fit within available hours.
4. Include breaks in the schedule.
5. Give priorities.
6. Tasks should be useful for the selected goal.
7. Do not create unnecessary work.

"""

with st.spinner(
    "🌀 AI is creating your productivity plan..."
):
    response = ask_groq(prompt, json_mode=True)

```

```

try:
    if response.startswith("AI Error:"):
        raise RuntimeError(response)

    plan = json.loads(response)

    for required_key in ("tasks", "activities", "schedule", "tips"):
        if required_key not in plan:
            raise ValueError(
                f"AI response is missing '{required_key}'."
            )

    st.session_state.generated_plan = plan

    # Add generated tasks
    for task in plan["tasks"]:
        st.session_state.tasks.append(
            {
                "name": task["name"],
                "description": task["description"],
                "priority": task["priority"],
                "hours": task["duration"] / 60,
                "category": task["category"],
                "deadline": str(deadline),
                "completed": False
            }
        )

    # Add generated activities
    for activity in plan["activities"]:

```

```

        st.session_state.activities.append(
            {
                "type": activity["type"],
                "topic": activity["name"],
                "duration": activity["duration"],
                "mood": "☐ AI Generated",
                "time": datetime.now().strftime(
                    "%d %b %Y, %I:%M %p"
                )
            }
        )

    st.success("☐ AI Productivity Plan Generated!")

except (json.JSONDecodeError, KeyError, TypeError, ValueError, RuntimeError) as e:
    st.error(
        f"Plan processing error: {e}"
    )

# Show generated plan
if st.session_state.generated_plan:
    plan = st.session_state.generated_plan
    st.divider()
    st.markdown(
        '<div class="section-title">☐ AI Generated Tasks</div>',
        unsafe_allow_html=True
    )
    for task in plan["tasks"]:

```



```

        for tip in plan["tips"]:

            st.info(

                f"❑ {tip}"

            )

# =====

# TASKS

# =====

elif menu == "❑ Tasks":

    st.markdown(

        '<div class="hero-title">❑ My Tasks</div>',

        unsafe_allow_html=True

    )

    if not st.session_state.tasks:

        st.info(

            "No tasks yet. Generate a plan from AI Productivity."

        )

    else:

        for i, task in enumerate(

            st.session_state.tasks

        ):

            if task["completed"]:

                icon = "❑"

            else:

                icon = "❑"

```

```
st.markdown(
    f"""
<div class="task-card">

    <h3>

        {icon} {task["name"]}

    </h3>

    <p>

        {task.get("description", "")}

    </p>

    <b><input type="checkbox"/> Priority:</b>

        {task["priority"]}

        &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~

    <b><input type="checkbox"/> Time:</b>

        {task["hours"]} hrs

        &nbsp;&nbsp;&nbsp;&nbsp;&~

    <b><input type="checkbox"/> Deadline:</b>

        {task["deadline"]}

    </div>

""",
    unsafe_allow_html=True
)

if not task["completed"]:
    if st.button(
        "<input type='checkbox'/'> Mark Complete",
        key=f"complete_{i}"
```

```

    ):

        st.session_state.tasks[i][

            "completed"

        ] = True

        st.rerun()

# =====

# ACTIVITIES

# =====

elif menu == "📅 Activities":

    st.markdown(

        '<div class="hero-title">📅 Study Activities</div>',

        unsafe_allow_html=True

    )

    st.write(

        "Activities generated by AI appear here."

    )

    if not st.session_state.activities:

        st.info(

            "No activities yet. Generate an AI productivity plan."

        )

    else:

        for activity in reversed(

            st.session_state.activities

        ):

```

```

st.markdown(
    f"""
    <div class="activity-card">

        <h3>

            □ {activity["topic"]}

        </h3>

        <p>

            □ Type:

                {activity["type"]}

        </p>

        <p>

            □ Duration:

                {activity["duration"]} minutes

        </p>

        <small>

            {activity["mood"]}

        </small>

    </div>

    """,
    unsafe_allow_html=True
)

```

```
# =====
```

```
# AI QUIZ
```

```
# =====
```

```

elif menu == "☐ AI Quiz":

    st.markdown(

        '<div class="hero-title">☐ AI Quiz</div>',

        unsafe_allow_html=True

    )

    c1, c2 = st.columns(2)

    with c1:

        subject = st.selectbox(

            "☐ Subject",

            [

                "Python",

                "DBMS",

                "Data Structures",

                "Artificial Intelligence",

                "Machine Learning",

                "Data Science"

            ]

        )

        topic = st.text_input(

            "☐ Topic",

            placeholder="Example: Python Functions"

        )

    with c2:

        difficulty = st.selectbox(

            "☐ Difficulty",

```

```

        [
            "Easy",
            "Medium",
            "Hard"
        ]
    )
    number = st.slider(
        "☐ Questions",
        3,
        10,
        5
    )
    if st.button("☐ Generate Quiz"):
        if not topic.strip():
            st.warning(
                "Please enter a topic."
            )
        else:
            prompt = f"""
Create {number} MCQ questions.
Subject: {subject}
Topic: {topic}
Difficulty: {difficulty}
Return ONLY JSON.
Format:

```

```

{{
"questions": [
  {{
    "question": "Question",
    "options": ["First option", "Second option", "Third option", "Fourth option"],
    "answer": "Exact text of the correct option",
    "explanation": "Short explanation"
  }}
]
}}

```

Rules:

Exactly {number} questions.

Exactly four options.

Only one correct answer. The answer field must exactly match the text of one option, not its letter.

"""

```

with st.spinner(
    "▢ AI is creating your quiz..."
):
    response = ask_groq(prompt, json_mode=True)
try:
    if response.startswith("AI Error:"):
        raise RuntimeError(response)
    quiz_data = json.loads(response)
    questions = quiz_data.get("questions")

```

```

if not isinstance(questions, list) or len(questions) != number:
    raise ValueError(
        f"AI must return exactly {number} questions."
    )
for question_data in questions:
    if (
        not isinstance(question_data, dict)
        or not isinstance(question_data.get("question"), str)
        or not isinstance(question_data.get("options"), list)
        or len(question_data["options"]) != 4
        or not all(isinstance(option, str) for option in question_data["options"])
        or not isinstance(question_data.get("explanation"), str)
    ):
        raise ValueError(
            "AI returned a question with an invalid format."
        )
    answer = question_data.get("answer")
    if not isinstance(answer, str):
        raise ValueError(
            "AI returned an answer that is not text."
        )
    answer = answer.strip()
    answer_labels = {"A": 0, "B": 1, "C": 2, "D": 3}
    if answer.upper() in answer_labels:
        answer = question_data["options"][answer_labels[answer.upper()]]

```

```

else:
    matching_option = next(
        (
            option for option in question_data["options"]
            if option.strip().casefold() == answer.casefold()
        ),
        None
    )
    if matching_option is None:
        raise ValueError(
            "AI answer must match one of the four options."
        )
    answer = matching_option
    question_data["answer"] = answer
    st.session_state.quiz = questions
    st.session_state.quiz_submitted = False
    st.session_state.quiz_score = None
    st.success("Quiz generated! 🎉")
except (json.JSONDecodeError, AttributeError, TypeError, ValueError, RuntimeError)
as e:
    st.error(
        f"Quiz error: {e}"
    )
if st.session_state.quiz:
    st.divider()

```

```

answers = {}

for i, q in enumerate(
    st.session_state.quiz
):
    st.markdown(
        f"### Q{i + 1}. {q['question']}"
    )
    answers[i] = st.radio(
        "Choose answer:",
        q["options"],
        key=f"q_{i}"
    )

if st.button("📝 Submit Quiz"):
    score = 0

    for i, q in enumerate(
        st.session_state.quiz
    ):
        if answers[i] == q["answer"]:
            score += 1

    st.session_state.quiz_score = score
    st.session_state.quiz_submitted = True
    st.rerun()

if st.session_state.quiz_submitted:
    score = st.session_state.quiz_score
    total = len(

```

```

        st.session_state.quiz
    )

    percentage = int(
        (score / total) * 100
    )

    st.subheader("📊 Your Result")

    c1, c2 = st.columns(2)

    c1.metric(
        "Score",
        f"{score}/{total}"
    )

    c2.metric(
        "Percentage",
        f"{percentage}%"
    )

    st.progress(
        percentage / 100
    )

    st.subheader(
        "📋 Answer Review"
    )

    for i, q in enumerate(
        st.session_state.quiz
    ):

```

```

        st.write(

            f"***Q{i + 1}. {q['question']}***"

        )

        st.write(

            f"☐ Correct Answer: {q['answer']}"

        )

        st.caption(

            f"☐ {q['explanation']}"

        )

# =====

# AI ASSISTANT

# =====

elif menu == "☐ AI Assistant":

    st.markdown(

        '<div class="hero-title">☐ AI Study Assistant</div>',

        unsafe_allow_html=True

    )

    st.markdown(

        """

        <div class="ai-card">

            <h2>☐ Ask StudyFlow AI</h2>

            <p>

                Ask questions about programming,

                academics, exams and interviews.

            </p>

        </div>

        """
    )

```

```

        </p>
    </div>

    """
    unsafe_allow_html=True
)

question = st.text_area(
    "❑ Ask your question",
    placeholder="Example: Explain DBMS normalization simply.",
    height=150,
    key="assistant_question"
)

c1, c2, c3, c4 = st.columns(4)

c1.button(
    "❑ Python",
    on_click=set_assistant_question,
    args=("Explain Python functions simply.",)
)

c2.button(
    "❑ DBMS",
    on_click=set_assistant_question,
    args=("Explain DBMS normalization simply.",)
)

c3.button(
    "❑ AI",
    on_click=set_assistant_question,

```

```

        args=("Explain Artificial Intelligence simply.",)
    )
c4.button(
    "❏ DSA",
    on_click=set_assistant_question,
    args=("Explain arrays and linked lists.",)
)
if st.button("❏ Ask AI"):
    if not question.strip():
        st.warning(
            "Please enter your question."
        )
    else:
        prompt = f"""
Student question:
{question}

Answer in a simple student-friendly way.

Include:

1. Simple explanation
2. Example
3. Important points
4. Exam/interview tip

"""

        with st.spinner(
            "❏ AI is thinking..."

```

```

    ):
        st.session_state.assistant_answer = ask_groq(prompt)
if st.session_state.assistant_answer is not None:
    answer = st.session_state.assistant_answer
    if answer.startswith("AI Error:"):
        st.error(answer)
    elif not answer.strip():
        st.error(
            "The AI returned an empty response. Please try again."
        )
    else:
        st.subheader("□ AI Explanation")
        st.markdown(answer)

# =====
# FOOTER
# =====

st.markdown(
    """
    <br><br>
    <div style="
        text-align:center;
        color:#8b8b9d;
        padding:20px;
    ">

```

□ StudyFlow AI

 •

Powered by Groq + Streamlit

<small>

Learn smarter. Plan better. Achieve more. □

</small>

</div>

""",

unsafe_allow_html=True

)